Database Systems Design I

Ground to Mastery — Study Notes

From zero knowledge to exam-ready — every concept explained and practiced

CSC 123 | Year 1 Semester 2 | DCS

Coursework 40% (Test 10% + Assignment 10% + Practical 20%) | Final Exam 60%

How to Study This Course

Database Systems is both theory and practical. You must understand the concepts AND be able to write SQL. The exam will ask you to design databases (draw ER diagrams, normalise tables) AND write SQL queries. Practice both equally.

> ⚡ *Install MySQL or PostgreSQL on your machine and practice every SQL example. Run it, modify it, break it, fix it — just like you do with C.*

> 📝 *Assessment: Test 10% + Assignment 10% + Practical 20% + Exam 60%. SQL writing, ER diagrams, and normalisation are the biggest exam topics.*

Install and start your database — practice alongside these notes

```
# Install MySQL on Ubuntu
sudo apt update && sudo apt install mysql-server

# Start MySQL
sudo systemctl start mysql

# Login
sudo mysql -u root

# Or install SQLite (simpler, no server needed)
sudo apt install sqlite3
sqlite3 mydb.db
```

UNIT 1: INTRODUCTION TO DATABASES

What databases are, why they exist, and core terminology

▸ 1.1 — What is a Database?

A database is an organised collection of structured data stored electronically, designed to be easily accessed, managed, and updated. It is managed by a Database Management System (DBMS).

Term / Concept	Definition / Explanation
`Database`	Organised collection of related data stored and accessed electronically
`DBMS`	Database Management System — software that manages the database — MySQL, PostgreSQL, Oracle, SQLite
`Data`	Raw facts — numbers, text, dates — unprocessed
`Information`	Processed, meaningful data — what you get after querying
`Database system`	The combination of the database + the DBMS + the applications using it
`Table`	A collection of related data in rows and columns — like a spreadsheet
`Record / Row`	One complete entry in a table — one student, one product, one order
`Field / Column`	One attribute of the data — name, age, price, date
`Primary Key`	A field that uniquely identifies each record — no duplicates, no NULL
`Foreign Key`	A field in one table that refers to the primary key of another table

▸ 1.2 — File System vs Database

Term / Concept	Definition / Explanation
`File system problem 1`	Data redundancy — same data stored in multiple files — wastes space
`File system problem 2`	Data inconsistency — different files have different values for same data
`File system problem 3`	Difficult to share data between programmes
`File system problem 4`	No security — anyone can open the file
`File system problem 5`	No concurrent access — two people cannot update same file safely
`Database solution`	Centralised data, no redundancy, controlled access, concurrent users, data integrity

▸ 1.3 — Advantages of a DBMS

1. Reduced data redundancy — data stored once, referenced everywhere
2. Data consistency — one update reflects everywhere
3. Data sharing — multiple users and applications access same data
4. Data security — access control, permissions, encryption
5. Data integrity — rules enforced — cannot enter invalid data
6. Backup and recovery — automatic backup, crash recovery
7. Concurrent access — multiple users can read/write simultaneously

▸ 1.4 — Types of Databases

Term / Concept	Definition / Explanation
`Relational (RDBMS)`	Data in tables with relationships — SQL — MySQL, PostgreSQL, Oracle, SQLite — most common
`NoSQL`	Non-relational — documents, key-value, graphs — MongoDB, Redis, Cassandra
`Hierarchical`	Tree structure — parent-child — IBM IMS — old, rarely used now
`Network`	Records linked in network — more flexible than hierarchical — old
`Object-oriented`	Data stored as objects — used with OOP languages
`NewSQL`	Relational + NoSQL scalability — Google Spanner, CockroachDB

▸ 1.5 — Database Users

Term / Concept	Definition / Explanation
`Database Administrator (DBA)`	Creates and maintains the database — manages security, backup, performance
`Application developer`	Writes programmes that use the database — uses SQL in code
`End user`	Uses applications that interact with the database — may not know SQL exists
`Data analyst`	Queries the database for reports and insights — writes complex SQL

UNIT 2: DATABASE MODELS & ARCHITECTURE

Relational model, ANSI/SPARC architecture, schemas

▸ 2.1 — The Relational Model

Proposed by E.F. Codd in 1970. Data is organised into relations (tables). Each relation has a schema (structure) and an instance (current data). All modern databases use this model.

Term / Concept	Definition / Explanation
`Relation`	A table — has a name, attributes (columns), and tuples (rows)
`Attribute`	A column — has a name and a domain (allowed values)
`Tuple`	A row — one record — one complete set of attribute values
`Domain`	The set of allowed values for an attribute — e.g. age must be 0-150
`Degree`	Number of attributes (columns) in a relation
`Cardinality`	Number of tuples (rows) in a relation
`Schema`	The structure/design of the table — column names and types
`Instance`	The actual data in the table at a point in time

▸ 2.2 — ANSI/SPARC Three-Level Architecture

This architecture separates how data is stored from how users see it — called data independence.

Term / Concept	Definition / Explanation
`External level`	What individual users or applications see — their view of the data — different users see different views
`Conceptual level`	The overall logical structure of the whole database — all tables and relationships — DBA works here
`Internal level`	How data is physically stored on disk — file organisation, indexes — hidden from users
`Logical independence`	Can change conceptual schema without affecting external views
`Physical independence`	Can change internal storage without affecting conceptual schema

▸ 2.3 — Keys in the Relational Model

Term / Concept	Definition / Explanation
`Super key`	Any set of attributes that uniquely identifies a tuple
`Candidate key`	Minimal super key — no attribute can be removed and still uniquely identify
`Primary key`	The chosen candidate key — uniquely identifies each row — NOT NULL — no duplicates

`Alternate key`	A candidate key that was NOT chosen as primary key
`Foreign key`	Attribute(s) that reference the primary key of another table — enforces referential integrity
`Composite key`	A primary key made of TWO or more attributes combined
`Surrogate key`	Artificial key — auto-generated integer — not from real data — e.g. student_id AUTO_INCREMENT

> 📝 *Exam: Given a table, identify all candidate keys, choose the primary key, and explain why. Very common question.*

UNIT 3: ENTITY-RELATIONSHIP (ER) MODELLING

Designing databases visually — the most important design skill

▸ 3.1 — ER Model Concepts

The Entity-Relationship model is a high-level conceptual model used to design a database before creating it. You draw an ER diagram to plan the structure.

Term / Concept	Definition / Explanation
`Entity`	A real-world object or concept — Student, Course, Employee, Product
`Entity set`	A collection of similar entities — all Students, all Courses
`Attribute`	A property of an entity — Student has: name, age, regNo, email
`Relationship`	An association between two or more entities — Student ENROLLS IN Course
`Relationship set`	All instances of a relationship — all enrolments
`ER Diagram`	Visual representation of entities, attributes, and relationships

▸ 3.2 — ER Diagram Symbols

Term / Concept	Definition / Explanation
`Rectangle`	Entity — e.g. STUDENT, COURSE, LECTURER
`Ellipse / Oval`	Attribute — e.g. name, age, regNo
`Double ellipse`	Multi-valued attribute — an attribute with multiple values — e.g. phone_numbers
`Dashed ellipse`	Derived attribute — calculated from other attributes — e.g. age from date_of_birth
`Diamond`	Relationship — e.g. ENROLLS, TEACHES, WORKS_FOR
`Double rectangle`	Weak entity — cannot exist without another entity — e.g. DEPENDENT of EMPLOYEE
`Double diamond`	Identifying relationship — connects weak entity to its owner
`Underline`	Primary key attribute — underlined in ellipse
`Line`	Connects entity to relationship or attribute to entity

▸ 3.3 — Cardinality (Relationship Types)

Term / Concept	Definition / Explanation
`One-to-One (1:1)`	One entity relates to exactly one other — e.g. one Employee has one Passport
`One-to-Many (1:N)`	One entity relates to many others — e.g. one Department has many Employees

`Many-to-One (N:1)`	Many entities relate to one — e.g. many Students belong to one Course
`Many-to-Many (M:N)`	Many relate to many — e.g. Students enrol in many Courses, Courses have many Students

▸ 3.4 — Participation Constraints

Term / Concept	Definition / Explanation
`Total participation`	Every entity MUST participate in the relationship — shown with double line
`Partial participation`	Some entities may not participate — shown with single line
`Example`	Every Employee MUST work for a Department (total) but a Department MAY have no projects (partial)

▸ 3.5 — Types of Attributes

Term / Concept	Definition / Explanation
`Simple`	Cannot be divided — age, salary, studentID
`Composite`	Can be divided into parts — full_name (first + last), address (street + city + country)
`Single-valued`	One value per entity — studentID, date_of_birth
`Multi-valued`	Multiple values — phone_numbers, email_addresses — shown with double ellipse
`Derived`	Calculated from other data — age (from DOB), total_price (from unit_price × quantity)
`Stored`	The base attribute a derived one comes from — date_of_birth stores, age derives
`Key attribute`	Uniquely identifies entity — shown underlined in ER diagram
`NULL`	Attribute has no value or unknown value for this entity

▸ 3.6 — Converting ER Diagram to Tables

1. Each entity becomes a table
2. Each simple attribute becomes a column
3. The key attribute becomes the primary key
4. For 1:N relationship — add foreign key to the 'many' side table
5. For M:N relationship — create a new junction/bridge table with both foreign keys
6. For 1:1 relationship — add foreign key to either side (usually the weaker side)
7. Multi-valued attributes — create a separate table

> 📝 *Exam: You will be given a scenario like 'A university has students who enrol in courses taught by lecturers' — draw the ER diagram AND convert it to tables. Practice this with different scenarios.*

UNIT 4: NORMALISATION

Eliminating redundancy and anomalies — 1NF, 2NF, 3NF, BCNF

▸ 4.1 — Why Normalise?

Without normalisation, databases have problems called anomalies that cause data corruption and inconsistency.

Term / Concept	Definition / Explanation
`Insertion anomaly`	Cannot add data without adding other unrelated data — e.g. cannot add a course until a student enrols
`Update anomaly`	Changing one piece of data requires updating many rows — miss one and data is inconsistent
`Deletion anomaly`	Deleting one record accidentally deletes other important data
`Normalisation`	The process of organising tables to reduce redundancy and eliminate anomalies
`Functional dependency`	Attribute B is functionally dependent on A if each value of A determines exactly one value of B — written A → B
`Partial dependency`	Non-key attribute depends on PART of a composite primary key — bad — causes 2NF violation
`Transitive dependency`	Non-key attribute depends on another non-key attribute — bad — causes 3NF violation

▸ 4.2 — First Normal Form (1NF)

A table is in 1NF if: all attributes contain atomic (indivisible) values, all values in a column are of the same type, each row is unique (has a primary key), and there are no repeating groups.

1NF Example

```
-- VIOLATES 1NF — multiple values in one cell
StudentID | Name     | Courses
1         | Bravehart| C, PHP, HTML     -- BAD: multiple values

-- IN 1NF — atomic values, separate rows
StudentID | Name     | CourseID
1         | Bravehart| C001
1         | Bravehart| C002
1         | Bravehart| C003
```

▸ 4.3 — Second Normal Form (2NF)

A table is in 2NF if: it is in 1NF, AND every non-key attribute is FULLY functionally dependent on the ENTIRE primary key (no partial dependencies). Only applies when primary key is composite.

2NF Example

```
-- VIOLATES 2NF — CourseName depends only on CourseID
-- Primary key is (StudentID, CourseID)
StudentID | CourseID | CourseName | Grade
1         | C001     | C Prog     | A     -- CourseName depends only on
CourseID

-- IN 2NF — split into two tables
-- Table 1: Enrolment
StudentID | CourseID | Grade
1         | C001     | A

-- Table 2: Course
CourseID | CourseName
C001     | C Programming
```

▸ 4.4 — Third Normal Form (3NF)

A table is in 3NF if: it is in 2NF, AND no non-key attribute is transitively dependent on the primary key (no non-key attribute depends on another non-key attribute).

3NF Example

```
-- VIOLATES 3NF — DeptName depends on DeptID, not StudentID
StudentID | Name     | DeptID | DeptName
1         | Bravehart| D01    | Computer Science
-- DeptName depends on DeptID (transitive dependency)
-- StudentID → DeptID → DeptName

-- IN 3NF — split into two tables
-- Table 1: Student
StudentID | Name     | DeptID
1         | Bravehart| D01

-- Table 2: Department
DeptID | DeptName
D01    | Computer Science
```

▸ 4.5 — Boyce-Codd Normal Form (BCNF)

A stronger version of 3NF. A table is in BCNF if: for every functional dependency A → B, A must be a super key. Most tables in 3NF are also in BCNF.

Term / Concept	Definition / Explanation
1NF	Atomic values, no repeating groups, has primary key
2NF	1NF + no partial dependencies (non-key fully depends on whole key)
3NF	2NF + no transitive dependencies (non-key depends only on key)
BCNF	3NF + every determinant is a super key — handles edge cases

	3NF misses	

> 📝 *Exam: Given an unnormalised table, normalise it to 3NF showing each step (UNF → 1NF → 2NF → 3NF). Identify and state each dependency that violates each normal form.*

UNIT 5: SQL — STRUCTURED QUERY LANGUAGE

The language of databases — DDL, DML, DQL, DCL

▸ 5.1 — SQL Categories

Term / Concept	Definition / Explanation
DDL — Data Definition Language	CREATE ALTER DROP — defines structure
DML — Data Manipulation Language	INSERT UPDATE DELETE — changes data
DQL — Data Query Language	SELECT — retrieves data
DCL — Data Control Language	GRANT REVOKE — controls access/permissions
TCL — Transaction Control Language	COMMIT ROLLBACK SAVEPOINT — manages transactions

▸ 5.2 — DDL — Creating Tables

DDL — create-tables.sql

```sql
-- Create database
CREATE DATABASE university;
USE university;

-- Create Department table
CREATE TABLE Department (
    dept_id     INT          PRIMARY KEY AUTO_INCREMENT,
    dept_name   VARCHAR(50)  NOT NULL UNIQUE,
    location    VARCHAR(100),
    created_at  DATE         DEFAULT (CURRENT_DATE)
);

-- Create Student table with foreign key
CREATE TABLE Student (
    student_id  INT          PRIMARY KEY AUTO_INCREMENT,
    reg_no      VARCHAR(20)  NOT NULL UNIQUE,
    first_name  VARCHAR(50)  NOT NULL,
    last_name   VARCHAR(50)  NOT NULL,
    email       VARCHAR(100) UNIQUE,
    dob         DATE,
    dept_id     INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);

-- Create Course table
CREATE TABLE Course (
    course_id   VARCHAR(10)  PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits     INT          NOT NULL CHECK (credits BETWEEN 1 AND 6),
```

```
    dept_id     INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

-- Junction table for M:N relationship
CREATE TABLE Enrolment (
    student_id  INT,
    course_id   VARCHAR(10),
    grade       CHAR(1)      CHECK (grade IN ('A','B','C','D','F')),
    enrol_date  DATE         DEFAULT (CURRENT_DATE),
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES Student(student_id)
        ON DELETE CASCADE,
    FOREIGN KEY (course_id) REFERENCES Course(course_id)
        ON DELETE CASCADE
);
```

▸ 5.3 — SQL Data Types

Term / Concept	Definition / Explanation
`INT / INTEGER`	Whole numbers — age, count, ID
`VARCHAR(n)`	Variable-length string up to n chars — names, emails
`CHAR(n)`	Fixed-length string — always n chars — grade, country code
`TEXT`	Long text — descriptions, comments — no length limit
`DATE`	Date only — YYYY-MM-DD — '2024-03-15'
`DATETIME`	Date and time — YYYY-MM-DD HH:MM:SS
`TIMESTAMP`	Date and time — auto-updates — used for created_at, updated_at
`DECIMAL(p,s)`	Precise decimal — p=total digits, s=decimal places — DECIMAL(10,2) for money
`FLOAT / DOUBLE`	Approximate decimal — scientific use — not for money
`BOOLEAN`	TRUE or FALSE — stored as TINYINT(1) in MySQL
`ENUM('a','b')`	One value from a fixed list — ENUM('M','F') for gender

▸ 5.4 — Constraints

Term / Concept	Definition / Explanation
`PRIMARY KEY`	Uniquely identifies row — NOT NULL + UNIQUE automatically
`FOREIGN KEY`	References primary key of another table — enforces referential integrity
`NOT NULL`	Column cannot be empty — must have a value
`UNIQUE`	All values in column must be different — can be NULL (unlike PK)

`CHECK`	Values must satisfy a condition — CHECK (age >= 18)
`DEFAULT`	Value used when none is provided — DEFAULT 0, DEFAULT 'active'
`AUTO_INCREMENT`	MySQL — automatically assigns next integer — used for surrogate keys

▸ 5.5 — DDL — Modifying and Dropping

DDL — alter-drop.sql

```
-- Add a column
ALTER TABLE Student ADD COLUMN phone VARCHAR(15);

-- Modify a column
ALTER TABLE Student MODIFY COLUMN email VARCHAR(150) NOT NULL;

-- Rename a column
ALTER TABLE Student RENAME COLUMN dob TO date_of_birth;

-- Drop a column
ALTER TABLE Student DROP COLUMN phone;

-- Add a constraint
ALTER TABLE Student ADD CONSTRAINT chk_email CHECK (email LIKE '%@%');

-- Drop a table
DROP TABLE Enrolment;

-- Drop database
DROP DATABASE university;

-- Remove all data but keep structure
TRUNCATE TABLE Enrolment;
```

▸ 5.6 — DML — Inserting Data

DML — insert-data.sql

```
-- Insert departments
INSERT INTO Department (dept_name, location) VALUES
    ('Computer Science', 'Block A'),
    ('Business', 'Block B'),
    ('Engineering', 'Block C');

-- Insert students
INSERT INTO Student (reg_no, first_name, last_name, email, dob, dept_id)
VALUES
    ('2024/DCS/001', 'Bravehart', 'TBH', 'bravehart@uni.ac.ug', '2004-05-15',
1),
    ('2024/DCS/002', 'Alice', 'Namukasa', 'alice@uni.ac.ug', '2003-11-20',
1),
```

```
    ('2024/BUS/001', 'Bob', 'Okello', 'bob@uni.ac.ug', '2004-02-10', 2);

-- Insert courses
INSERT INTO Course VALUES
    ('CSC122', 'Structured Software Development', 4, 1),
    ('CSC121', 'Computer Hardware', 4, 1),
    ('CSC123', 'Database Systems Design I', 4, 1);

-- Insert enrolments
INSERT INTO Enrolment (student_id, course_id, grade) VALUES
    (1, 'CSC122', 'A'),
    (1, 'CSC123', 'B'),
    (2, 'CSC122', 'A'),
    (2, 'CSC121', 'C');
```

▸ 5.7 — DML — Updating and Deleting

DML — update-delete.sql

```
-- Update a single record
UPDATE Student
SET email = 'newemail@uni.ac.ug'
WHERE student_id = 1;

-- Update multiple records
UPDATE Enrolment
SET grade = 'B'
WHERE student_id = 2 AND course_id = 'CSC121';

-- Delete a specific record
DELETE FROM Enrolment
WHERE student_id = 3 AND course_id = 'CSC122';

-- Delete all records for a student
DELETE FROM Student WHERE student_id = 3;

-- WARNING: DELETE without WHERE deletes ALL rows!
-- DELETE FROM Student;  -- deletes everything!
```

> ⚠️ *Always use WHERE with UPDATE and DELETE. Without WHERE, you modify or delete EVERY row. Always test with a SELECT first.*

UNIT 6: SQL — SELECT QUERIES

Retrieving data — the most used SQL skill

▸ 6.1 — Basic SELECT

select-basic.sql

```
-- Select all columns
SELECT * FROM Student;

-- Select specific columns
SELECT first_name, last_name, email FROM Student;

-- Column aliases
SELECT first_name AS 'First Name',
       last_name  AS 'Surname',
       dept_id    AS Department
FROM Student;

-- Calculated column
SELECT course_name, credits, credits * 15 AS 'Hours' FROM Course;

-- Remove duplicates
SELECT DISTINCT dept_id FROM Student;

-- Limit results
SELECT * FROM Student LIMIT 5;
SELECT * FROM Student LIMIT 5 OFFSET 10; -- skip first 10, show next 5
```

▸ 6.2 — WHERE Clause — Filtering

select-where.sql

```
-- Simple condition
SELECT * FROM Student WHERE dept_id = 1;

-- Comparison operators
SELECT * FROM Course WHERE credits >= 4;
SELECT * FROM Student WHERE last_name != 'Okello';

-- AND, OR, NOT
SELECT * FROM Student
WHERE dept_id = 1 AND last_name = 'TBH';

SELECT * FROM Course
WHERE credits = 3 OR credits = 4;

-- BETWEEN
SELECT * FROM Course WHERE credits BETWEEN 3 AND 5;

-- IN — match any value in list
SELECT * FROM Student WHERE dept_id IN (1, 2);
```

```sql
-- LIKE — pattern matching
SELECT * FROM Student WHERE email LIKE '%@uni.ac.ug';
SELECT * FROM Student WHERE first_name LIKE 'B%';  -- starts with B
SELECT * FROM Student WHERE first_name LIKE '%art'; -- ends with art
SELECT * FROM Student WHERE first_name LIKE '_r%';  -- second char is r

-- IS NULL / IS NOT NULL
SELECT * FROM Student WHERE email IS NULL;
SELECT * FROM Student WHERE email IS NOT NULL;
```

▸ 6.3 — ORDER BY and GROUP BY

select-groupby.sql

```sql
-- Sort ascending (default)
SELECT * FROM Student ORDER BY last_name;
SELECT * FROM Student ORDER BY last_name ASC;

-- Sort descending
SELECT * FROM Course ORDER BY credits DESC;

-- Sort by multiple columns
SELECT * FROM Student ORDER BY dept_id ASC, last_name ASC;

-- GROUP BY — aggregate per group
SELECT dept_id, COUNT(*) AS student_count
FROM Student
GROUP BY dept_id;

-- GROUP BY with HAVING — filter groups
SELECT dept_id, COUNT(*) AS student_count
FROM Student
GROUP BY dept_id
HAVING COUNT(*) > 1;

-- WHERE filters rows BEFORE grouping
-- HAVING filters groups AFTER grouping
```

▸ 6.4 — Aggregate Functions

Term / Concept	Definition / Explanation
`COUNT(*)`	Count all rows — COUNT(column) counts non-NULL values
`SUM(column)`	Total sum of a numeric column
`AVG(column)`	Average value of a numeric column
`MIN(column)`	Smallest value
`MAX(column)`	Largest value

aggregate-functions.sql

```sql
-- Count all students
SELECT COUNT(*) AS total_students FROM Student;

-- Average credits per department
SELECT dept_id, AVG(credits) AS avg_credits
FROM Course
GROUP BY dept_id;

-- Students per department with totals
SELECT d.dept_name, COUNT(s.student_id) AS students
FROM Department d
LEFT JOIN Student s ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY students DESC;
```

▸ 6.5 — JOINS — Combining Tables

joins.sql

```sql
-- INNER JOIN — only matching rows from both tables
SELECT s.first_name, s.last_name, d.dept_name
FROM Student s
INNER JOIN Department d ON s.dept_id = d.dept_id;

-- LEFT JOIN — all rows from left table, matching from right
SELECT s.first_name, d.dept_name
FROM Student s
LEFT JOIN Department d ON s.dept_id = d.dept_id;
-- Students with no department will show NULL for dept_name

-- RIGHT JOIN — all rows from right, matching from left
SELECT s.first_name, d.dept_name
FROM Student s
RIGHT JOIN Department d ON s.dept_id = d.dept_id;
-- Departments with no students will show NULL for first_name

-- Three-table join — student, enrolment, course
SELECT s.first_name, s.last_name, c.course_name, e.grade
FROM Student s
INNER JOIN Enrolment e ON s.student_id = e.student_id
INNER JOIN Course c    ON e.course_id  = c.course_id
ORDER BY s.last_name, c.course_name;
```

Term / Concept	Definition / Explanation
`INNER JOIN`	Returns rows where there is a match in BOTH tables — most common
`LEFT JOIN`	All rows from LEFT table — NULL for right side where no match
`RIGHT JOIN`	All rows from RIGHT table — NULL for left side where no match
`FULL OUTER JOIN`	All rows from BOTH tables — NULL where no match on either side

`CROSS JOIN`	Every combination of rows — rarely used — no ON condition
`SELF JOIN`	Table joins with itself — for hierarchical data like manager-employee

▸ 6.6 — Subqueries

subqueries.sql

```sql
-- Subquery in WHERE
SELECT first_name, last_name
FROM Student
WHERE dept_id = (
    SELECT dept_id FROM Department
    WHERE dept_name = 'Computer Science'
);

-- Subquery with IN
SELECT first_name, last_name
FROM Student
WHERE student_id IN (
    SELECT student_id FROM Enrolment
    WHERE grade = 'A'
);

-- Subquery in FROM (derived table)
SELECT dept_id, avg_grade
FROM (
    SELECT dept_id, AVG(credits) AS avg_grade
    FROM Course
    GROUP BY dept_id
) AS dept_stats
WHERE avg_grade > 3;

-- Correlated subquery — references outer query
SELECT first_name, last_name
FROM Student s
WHERE EXISTS (
    SELECT 1 FROM Enrolment e
    WHERE e.student_id = s.student_id
    AND e.grade = 'A'
);
```

UNIT 7: VIEWS, INDEXES & TRANSACTIONS

Database objects that improve security, speed and reliability

▸ 7.1 — Views

A view is a virtual table based on a SELECT query. It does not store data — it stores the query. Every time you query the view, it runs the underlying SELECT.

views.sql

```sql
-- Create a view
CREATE VIEW StudentCourses AS
SELECT s.student_id, s.first_name, s.last_name,
       c.course_name, e.grade
FROM Student s
JOIN Enrolment e ON s.student_id = e.student_id
JOIN Course c    ON e.course_id  = c.course_id;

-- Use it like a table
SELECT * FROM StudentCourses WHERE grade = 'A';

-- View showing only CS students
CREATE VIEW CS_Students AS
SELECT student_id, first_name, last_name, email
FROM Student
WHERE dept_id = 1;

-- Drop a view
DROP VIEW StudentCourses;

-- Update a view
CREATE OR REPLACE VIEW CS_Students AS
SELECT student_id, first_name, last_name
FROM Student WHERE dept_id = 1;
```

Term / Concept	Definition / Explanation
`View advantage 1`	Security — show only certain columns/rows to certain users
`View advantage 2`	Simplicity — complex JOIN queries become simple SELECT from view
`View advantage 3`	Consistency — underlying query runs fresh every time
`View limitation`	Cannot always INSERT/UPDATE through a view — especially with JOINs

▸ 7.2 — Indexes

An index speeds up data retrieval at the cost of extra storage and slower writes. Like a book index — instead of reading every page, you jump directly to the right page.

indexes.sql

```sql
-- Create index on frequently searched column
CREATE INDEX idx_student_email ON Student(email);

-- Create composite index
CREATE INDEX idx_enrol ON Enrolment(student_id, course_id);

-- Create unique index
CREATE UNIQUE INDEX idx_reg_no ON Student(reg_no);

-- Drop index
DROP INDEX idx_student_email ON Student;

-- See indexes on a table
SHOW INDEX FROM Student;

-- PRIMARY KEY automatically creates an index
-- UNIQUE constraint automatically creates an index
```

▸ 7.3 — Transactions

A transaction is a group of SQL operations that must ALL succeed or ALL fail together. If any step fails, all changes are rolled back. This ensures data integrity.

Term / Concept	Definition / Explanation
`ACID`	Atomicity, Consistency, Isolation, Durability — properties of a good transaction
`Atomicity`	All operations succeed or all are rolled back — no partial completion
`Consistency`	Database moves from one valid state to another
`Isolation`	Transactions don't interfere with each other
`Durability`	Committed data persists even after system crash
`COMMIT`	Make all changes permanent
`ROLLBACK`	Undo all changes back to last COMMIT or SAVEPOINT
`SAVEPOINT`	Create a restore point within a transaction

transactions.sql

```sql
-- Transaction example — transfer credits between accounts
START TRANSACTION;

-- Deduct from account 1
UPDATE Account SET balance = balance - 500 WHERE account_id = 1;

-- Add to account 2
UPDATE Account SET balance = balance + 500 WHERE account_id = 2;

-- If both succeeded, commit
```

```
COMMIT;

-- If anything went wrong
-- ROLLBACK;

-- With savepoint
START TRANSACTION;
INSERT INTO Enrolment VALUES (1, 'CSC122', 'A', NOW());
SAVEPOINT after_insert;
UPDATE Student SET dept_id = 2 WHERE student_id = 1;
ROLLBACK TO after_insert;  -- undo UPDATE but keep INSERT
COMMIT;
```

UNIT 8: DATABASE SECURITY & ADMINISTRATION

Users, permissions, backup, recovery

▸ 8.1 — Database Security

Term / Concept	Definition / Explanation
`Authentication`	Verifying identity — username and password to log in to DBMS
`Authorisation`	What an authenticated user is allowed to do — READ, WRITE, DELETE
`DCL — GRANT`	Give privileges to a user
`DCL — REVOKE`	Remove privileges from a user
`Role`	A named collection of privileges — assign role to user instead of individual privileges
`SQL Injection`	Attacker inserts malicious SQL into input fields — most common database attack

DCL — users-permissions.sql

```sql
-- Create a database user
CREATE USER 'appuser'@'localhost' IDENTIFIED BY 'strongpassword';

-- Grant specific privileges
GRANT SELECT, INSERT, UPDATE ON university.Student TO 'appuser'@'localhost';

-- Grant all privileges on a database
GRANT ALL PRIVILEGES ON university.* TO 'appuser'@'localhost';

-- Apply changes
FLUSH PRIVILEGES;

-- Show user privileges
SHOW GRANTS FOR 'appuser'@'localhost';

-- Revoke privileges
REVOKE DELETE ON university.Student FROM 'appuser'@'localhost';

-- Drop user
DROP USER 'appuser'@'localhost';
```

▸ 8.2 — SQL Injection — Know and Prevent

SQL Injection — understand this for exam

```sql
-- VULNERABLE query — never do this in real code
-- If user enters: ' OR '1'='1
-- The query becomes: SELECT * FROM users WHERE name='' OR '1'='1'
-- This returns ALL users — security breach!
```

```
-- SAFE — use prepared statements (in PHP)
-- $stmt = $pdo->prepare('SELECT * FROM users WHERE name = ?');
-- $stmt->execute([$username]);

-- Always validate and sanitise input
-- Never concatenate user input directly into SQL strings
```

▸ 8.3 — Backup and Recovery

Backup commands — run in your terminal

```
# Backup entire database
mysqldump -u root -p university > university_backup.sql

# Backup specific table
mysqldump -u root -p university Student > student_backup.sql

# Restore from backup
mysql -u root -p university < university_backup.sql

# Backup all databases
mysqldump -u root -p --all-databases > all_backup.sql
```

UNIT 9: COMPLETE PRACTICE DATABASE

Build a full university database from scratch

Practice this entire database in MySQL. Create it, insert data, run every query. This covers every topic in the course.

complete-practice.sql — run this in MySQL

```sql
-- COMPLETE UNIVERSITY DATABASE
-- Practice this entire script from top to bottom

CREATE DATABASE IF NOT EXISTS uni_practice;
USE uni_practice;

-- TABLES
CREATE TABLE Department (
    dept_id   INT         PRIMARY KEY AUTO_INCREMENT,
    dept_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE Lecturer (
    lec_id    INT          PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    email     VARCHAR(100) UNIQUE,
    dept_id   INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

CREATE TABLE Student (
    student_id INT          PRIMARY KEY AUTO_INCREMENT,
    reg_no     VARCHAR(20)  NOT NULL UNIQUE,
    full_name  VARCHAR(100) NOT NULL,
    email      VARCHAR(100) UNIQUE,
    dept_id    INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

CREATE TABLE Course (
    course_id   VARCHAR(10)  PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits     INT          CHECK (credits BETWEEN 1 AND 6),
    lec_id      INT,
    FOREIGN KEY (lec_id) REFERENCES Lecturer(lec_id)
);

CREATE TABLE Enrolment (
    student_id INT,
    course_id  VARCHAR(10),
    grade      CHAR(1),
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES Student(student_id) ON DELETE
CASCADE,
    FOREIGN KEY (course_id)  REFERENCES Course(course_id)  ON DELETE CASCADE
);
```

```sql
-- INSERT DATA
INSERT INTO Department (dept_name) VALUES
    ('Computer Science'), ('Business'), ('Engineering');

INSERT INTO Lecturer (full_name, email, dept_id) VALUES
    ('Dr. Mukasa', 'mukasa@uni.ac.ug', 1),
    ('Prof. Auma', 'auma@uni.ac.ug', 1);

INSERT INTO Student (reg_no, full_name, email, dept_id) VALUES
    ('2024/DCS/001', 'Bravehart TBH', 'bravehart@uni.ac.ug', 1),
    ('2024/DCS/002', 'Alice Namukasa', 'alice@uni.ac.ug', 1),
    ('2024/BUS/001', 'Bob Okello', 'bob@uni.ac.ug', 2);

INSERT INTO Course VALUES
    ('CSC122', 'Structured Software Dev', 4, 1),
    ('CSC121', 'Computer Hardware', 4, 1),
    ('CSC123', 'Database Systems I', 4, 2);

INSERT INTO Enrolment VALUES
    (1,'CSC122','A'), (1,'CSC123','B'),
    (2,'CSC122','A'), (2,'CSC121','C'),
    (3,'CSC122','B');

-- PRACTICE QUERIES
-- 1. All students with their department
SELECT s.full_name, d.dept_name FROM Student s
JOIN Department d ON s.dept_id = d.dept_id;

-- 2. Students and their courses and grades
SELECT s.full_name, c.course_name, e.grade
FROM Student s
JOIN Enrolment e ON s.student_id = e.student_id
JOIN Course c    ON e.course_id  = c.course_id
ORDER BY s.full_name;

-- 3. Count students per department
SELECT d.dept_name, COUNT(s.student_id) AS total
FROM Department d
LEFT JOIN Student s ON d.dept_id = s.dept_id
GROUP BY d.dept_id;

-- 4. Students who got grade A
SELECT DISTINCT s.full_name
FROM Student s
JOIN Enrolment e ON s.student_id = e.student_id
WHERE e.grade = 'A';

-- 5. Courses with more than 1 student enrolled
SELECT c.course_name, COUNT(*) AS enrolled
FROM Course c
JOIN Enrolment e ON c.course_id = e.course_id
GROUP BY c.course_id
HAVING COUNT(*) > 1;
```

> ✅ *Practice this entire script. Type every line — do not copy-paste. Modifying and re-running queries is how you truly learn SQL.*

SQL Quick Reference Card

SQL Quick Reference — memorise this

```
-- DDL
CREATE TABLE t (col TYPE CONSTRAINT, ...);
ALTER TABLE t ADD COLUMN col TYPE;
ALTER TABLE t MODIFY COLUMN col NEW_TYPE;
ALTER TABLE t DROP COLUMN col;
DROP TABLE t;
TRUNCATE TABLE t;

-- DML
INSERT INTO t (col1, col2) VALUES (v1, v2);
UPDATE t SET col = val WHERE condition;
DELETE FROM t WHERE condition;

-- DQL
SELECT col1, col2 FROM t
WHERE condition
GROUP BY col
HAVING group_condition
ORDER BY col ASC|DESC
LIMIT n OFFSET m;

-- JOINS
SELECT * FROM a INNER JOIN b ON a.id = b.a_id;
SELECT * FROM a LEFT  JOIN b ON a.id = b.a_id;
SELECT * FROM a RIGHT JOIN b ON a.id = b.a_id;

-- AGGREGATES
COUNT(*) SUM(col) AVG(col) MIN(col) MAX(col)

-- DCL
GRANT SELECT ON db.table TO 'user'@'host';
REVOKE SELECT ON db.table FROM 'user'@'host';

-- TRANSACTIONS
START TRANSACTION; ... COMMIT; / ROLLBACK;

-- VIEWS
CREATE VIEW v AS SELECT ...;
DROP VIEW v;
```